Archived LoRAMBo draft

The experimental tables in the following historical draft are withdrawn. They are not backed by verified runs. The old allocation guarantees also require the corrections explained in the current working note.

The original pages follow unchanged so the earlier claims remain identifiable. Use lora_master.pdf and its accompanying verification script for the corrected account.

Correction recorded October 2026.

LoRAMBo: Fighting LoRA Memory Bottlenecks With Optimized Rank Selection

Liam Cawley*
Department of Computer Science
University of Michigan
Ann Arbor, MI 48103
cawleyl@umich.edu

Abstract

Low-Rank Adaptation (LoRA) efficiently fine-tunes large pre-trained language models through low-rank weight updates, significantly reducing memory usage compared to full fine-tuning. However, the problem of *how to optimally allocate low-rank parameters across model layers* remains challenging, and prior strategies largely relied on heuristics.

In this paper, we present an extended theoretical framework that unifies **(1)** classical matrix approximation perspectives (via the Eckart–Young–Mirsky theorem), **(2)** data- and curvature-aware approaches that leverage Hessian-based sensitivity measures, **(3)** online rank-allocation algorithms with near-optimality guarantees under a global parameter budget, and **(4)** empirical validation on large-scale NLP tasks.

First, we show that optimal rank allocation (under a Frobenius-norm view) scales with layer sensitivity and the inverse of layer dimension. We prove error bounds, convergence guarantees, and memory complexity results that guide practical adaptive methods. Next, we refine these results by incorporating Hessian-based sensitivity metrics, thereby grounding "layer importance" in second-order curvature. We propose new data-dependent norms to further tighten approximation bounds when the data manifold resides in a low-rank subspace. Finally, we develop an online rank-allocation algorithm that iteratively chooses rank increments per layer, and prove that it achieves near-optimal performance under mild Lipschitz and submodularity-like assumptions. Extensive experiments on GLUE benchmark tasks (QNLI, SST-2, MNLI), WikiText-103, and ImageNet confirm that our theoretical insights yield improved performance–memory trade-offs relative to uniform or purely heuristic-based methods. Our results significantly advance the theoretical understanding of efficient model adaptation and provide strong empirical evidence for adopting curvature- and data-aware rank selection strategies in large-scale applications.

1 Introduction and Background

Fine-tuning large pre-trained language models has become standard practice for NLP tasks, yet the process remains computationally expensive and memory-intensive. Low-Rank Adaptation (LoRA) [7] reduces trainable parameters through rank-constrained updates to specific layers, typically expressed as

$$W_i + A_i B_i \tag{1}$$

where $W_i \in \mathbb{R}^{d_i \times k_i}$ is the weight matrix for layer i, and $A_i \in \mathbb{R}^{d_i \times r_i}$, $B_i \in \mathbb{R}^{r_i \times k_i}$ are low-rank factors. While LoRA is effective in saving parameters and memory, the question of *how to distribute*

*Equal contribution

38th Conference on Neural Information Processing Systems (NeurIPS 2024).

the total "rank budget" across layers remains relatively unexplored. Different layers often exhibit varying sensitivities, but most existing strategies rely on uniform or ad-hoc rank allocations.

Motivation. Empirical findings from prior work indicate that not all layers in a large language model are equally important for a given downstream task [12, 4, 13], and singular value distributions can vary significantly across layers. This strongly suggests that a *layer-wise adaptive* rank assignment can yield better approximation and faster training, while still adhering to a global memory budget. However, robust theoretical frameworks for analyzing or guiding these decisions have been limited.

Contributions. We offer several complementary theoretical and practical advances:

1. We present a **theoretical framework** establishing error bounds, optimal rank distributions, and convergence guarantees for LoRA based on matrix approximation theory (Eckart–Young–Mirsky) and SGD convergence. In particular, we show that *optimal rank allocation scales with layer sensitivity and inversely with layer dimension*.
2. We extend this analysis with **Hessian-based** or **data-dependent** sensitivity measures. This links LoRA rank allocation to second-order curvature and data manifold properties, providing refined (and often tighter) error guarantees.
3. We propose an **online rank-allocation** algorithm for distributing rank increments across layers during training, and prove it achieves near-optimal solutions under mild Lipschitz and submodularity-like conditions.
4. We validate these methods on three GLUE benchmark tasks (QNLI, SST-2, MNLI) across three model families (BERT, RoBERTa, T5), as well as on WikiText-103 (language modeling) and ImageNet-1k (vision). We consistently observe that sensitivity-guided rank adaptation (whether via Hessian-based or data-based measures) reduces the performance gap with full fine-tuning at minimal memory overhead.

1.1 Related Work and Positioning

LoRA was originally proposed by Hu et al. [7] as a lightweight adaptation method that injects low-rank A_iB_i factors into certain layers of a large model. Follow-up works extended the idea [4, 13], e.g. by dynamically adjusting each layer's rank. However, a unifying theory bridging matrix approximations, second-order curvature, and online knapsack-like rank selection was lacking. We build on classic matrix approximation theory [6], as well as standard results in SGD optimization [2], to derive new performance bounds. We further incorporate Hessian-based curvature approximations (similar to K-FAC-like methods [10]) and submodular streaming/knapsack arguments [1]. Our results also complement other parameter-efficient fine-tuning approaches such as adapters [14] or prefix tuning [8], which could likewise benefit from theory-based parameter allocation.

2 Theoretical Framework

We now merge and expand the theoretical treatments from prior sections and newly developed approaches, providing a comprehensive view of LoRA rank allocation. This includes classical low-rank matrix approximation bounds, Hessian-based and data-dependent error measures, as well as an online rank-allocation algorithm with near-optimal guarantees.

2.1 Problem Setup and Notation

Consider a pre-trained model with L layers. Layer i has weight matrix $W_i \in \mathbb{R}^{d_i \times k_i}$. LoRA introduces low-rank factors $A_i \in \mathbb{R}^{d_i \times r_i}$ and $B_i \in \mathbb{R}^{r_i \times k_i}$, yielding the adapted weight $W_i + A_iB_i$. The total parameter budget constraint for these low-rank factors is:

$$\sum_{i=1}^{L} r_i(d_i + k_i) \;\leq\; P_{max}.$$

Denote by $s_i > 0$ a *layer sensitivity* parameter reflecting how important layer i is for the end-to-end loss (or equivalently, how large the penalty is when W_i is poorly approximated). This s_i could be set in various ways:

- **Gradient-based:** e.g. average squared gradient magnitude of layer i [12].
- **Hessian-based:** e.g. the trace or operator norm of the Hessian block for layer i (see Sec. 2.4).
- **Data-based:** e.g. measuring the data covariance for transformations at layer i (see Sec. 2.5).

2.2 Classical Frobenius-Norm Bounds and Optimal Allocation

We begin with classical matrix approximation results. Let W_i have singular value decomposition $W_i = U_i \Sigma_i V_i^\top$, with descending singular values $\sigma_1 \geq \sigma_2 \geq \dots$. A rank-$r_i$ approximation $A_i B_i$ that *minimizes* $\|W_i - A_i B_i\|_F^2$ is obtained by truncating the top r_i singular values, yielding the well-known Eckart–Young–Mirsky theorem:

Lemma 1 (Best Rank-r Approximation, [6])**.**

$$\min_{\mathrm{rank}(X) \leq r_i} \|W_i - X\|_F^2 \;=\; \sum_{j=r_i+1}^{\min(d_i,k_i)} \sigma_j^2(W_i).$$

We incorporate layer sensitivity s_i by considering the objective:

$$\min_{\{r_i\}} \sum_{i=1}^{L} s_i \|W_i - A_i B_i\|_F^2 \quad \text{subject to} \quad \sum_{i=1}^{L} r_i (d_i + k_i) \leq P_{\max}.$$

The *optimal* rank distribution $\{r_i^*\}$ in the sense of a Lagrangian relaxation can be approximated in closed form:

Theorem 1 (Optimal Rank Distribution (Frobenius View))**.** *Under the parameter budget $P_{\max}$, the approximately optimal rank allocation that minimizes $\sum_{i=1}^{L} s_i \|W_i - A_i B_i\|_F^2$ obeys*

$$r_i^* \;\approx\; \left\lceil \frac{P_{\max}}{L} \cdot \frac{s_i^{1/2} \, (d_i + k_i)^{-1/2}}{\sum_{j=1}^{L} s_j^{1/2} \, (d_j + k_j)^{-1/2}} \right\rceil.$$

Sketch of Proof. See Appendix **??**. Form the Lagrangian

$$\mathcal{L}(\{r_i\}, \lambda) \;=\; \sum_{i=1}^{L} s_i \|W_i - A_i B_i\|_F^2 \;+\; \lambda \Big(\sum_{i=1}^{L} r_i (d_i + k_i) - P_{\max} \Big).$$

Using standard matrix approximation bounds from Lemma 1 and differentiating w.r.t. r_i leads to the proportionality $r_i \propto s_i^{1/2} (d_i + k_i)^{-1/2}$, which is then normalized to meet the budget constraint. □

Interpretation. This reveals that layers with larger sensitivity s_i should receive more rank, and that bigger layers (large d_i, k_i) receive proportionally *smaller* rank. The net effect is a "square-root law" reminiscent of many resource allocation problems.

2.3 Frobenius-Norm Convergence and Memory Bounds

Theorem 2 (Convergence Rate under SGD)**.** *Assume standard SGD conditions (bounded gradients, diminishing step sizes, etc.). Then for rank-constrained updates $\{A_i, B_i\}$ of dimension $\sum_i r_i (d_i + k_i) \leq P_{\max}$, the training convergence rate satisfies:*

$$\mathbb{E}\big[\mathcal{L}(\theta_T) - \mathcal{L}^*\big] \;\leq\; \mathcal{O}\Big(\frac{1}{\sqrt{T}} \sum_{i=1}^{L} \frac{s_i}{r_i} \Big).$$

In particular, higher ranks in sensitive layers reduce the constant factor in the $\frac{1}{\sqrt{T}}$ convergence bound.

Sketch. Under rank constraints, effectively the parameter space is of dimension $\sum_i r_i(d_i + k_i)$. Standard SGD analyses (e.g. [2]) give

$$\mathcal{L}(\theta_{t+1}) - \mathcal{L}(\theta^*) \;\le\; \|\theta_{t+1} - \theta^*\|^2 - \|\theta_t - \theta^*\|^2 + \dots$$

plus a penalty for the approximation error from limiting each W_i to rank r_i. That penalty is smaller if r_i is bigger for layers with large s_i. Detailed steps appear in Appendix **??**. □

Proposition 1 (Memory Bound)**.** *If each parameter uses b bits, the total memory usage of LoRA factors satisfies*

$$M_{\text{total}} \;\le\; P_{\max} \cdot b \;+\; \mathcal{O}(L).$$

Hence, controlling $P_{\max}$ directly caps memory usage.

2.4 Hessian-Based Sensitivity

Layer sensitivity s_i can be assigned more accurately by approximating second-order curvature. Let H_i be the Hessian of the loss restricted to layer i. Define

$$s_i \;=\; \text{Tr}(H_i) \quad \text{or} \quad s_i \;=\; \|H_i\|_{\text{op}},$$

reflecting how changes in W_i affect the overall loss. One obtains a refined bound:

Theorem 3 (Hessian-Based Allocation)**.** *Let f_θ be an L-layer neural network with parameters $\theta = \{W_i\}$, trained with loss ℓ. Suppose the Hessian block for layer i satisfies $\nabla^2_{W_i}\ell \preceq H_i$ in expectation over data. Then for a rank-r_i approximation of W_i with $\|\Delta W_i\|_F = \|W_i - A_iB_i\|_F$, the corresponding loss increase satisfies*

$$\mathbb{E}[\Delta\ell] \;\le\; c_i \,\text{Tr}(H_i)\, \|\Delta W_i\|_F^2,$$

where c_i depends on Lipschitz constants of subsequent layers. Minimizing $\sum_i s_i \|\Delta W_i\|_F^2$ with $s_i = \text{Tr}(H_i)$ (or $\|H_i\|_{\text{op}}$) thus minimizes a bound on the total end-to-end loss.

Outline. A second-order Taylor expansion around the (locally) optimal W_i implies that the first-order term vanishes in expectation. Hence the dominant term is $\Delta W_i^\top H_i \, \Delta W_i$, which is bounded by $\text{Tr}(H_i)\|\Delta W_i\|_F^2$. An additional factor c_i accounts for error propagation through subsequent layers; see detailed discussion in the new theoretical findings (Section 3 of the appended text). □

Practical Implication. Instead of a purely Frobenius-based measure, layers with higher Hessian trace (i.e. higher curvature) are penalized more. This can lead to more accurate rank allocations, especially in large transformer architectures where some layers exhibit significantly higher curvature than others.

2.5 Data-Dependent Error Measures

Classical LoRA approximations measure $\|W_i - A_iB_i\|_F$ in isolation. In practice, it may suffice to approximate W_i accurately *only* in directions frequently encountered by the data distribution. For instance, if $G_i = \mathbb{E}_x[\Psi_i(x)\Psi_i(x)^\top]$ is the data covariance at layer i, we can measure approximation quality via

$$\text{Tr}\big((W_i - A_iB_i)^\top G_i (W_i - A_iB_i)\big).$$

Such a measure can be strictly smaller than the naive Frobenius norm if W_i has large singular values in directions orthogonal to typical data inputs. This yields sharper bounds [1] for the actual *training* or *inference* loss.

Proposition 2 (Data-Weighted LoRA Error)**.** *Here, G_i is a positive semi-definite matrix derived from layer i's activations (e.g., an empirical covariance). Then the best rank-r_i approximation of W_i w.r.t. the data-weighted norm*

$$\|W_i - X\|_{G_i}^2 \;=\; \text{Tr}\big((W_i - X)^\top G_i (W_i - X)\big)$$

may differ from the standard top-r_i singular value truncation w.r.t. the usual Frobenius norm. If G_i strongly down-weights certain directions, the effective rank needed can be lower, improving the memory–accuracy tradeoff for real data.

Summary. In short, layering Hessian or data covariance weighting on top of the classical $\|W_i - A_i B_i\|_F$ measure can further refine rank allocations to reflect the *true* importance of each dimension for the end-task.

2.6 An Online Rank-Allocation Algorithm with Near-Optimality Guarantees

Beyond closed-form approximations, *incremental* or *online* methods can adapt ranks as training progresses. We now sketch an algorithmic procedure that repeatedly estimates the marginal benefit of adding 1 unit of rank to each layer, picking whichever layer yields the best ratio of "improvement per parameter cost." We prove that, under mild assumptions, this greedy approach is near-optimal.

Algorithmic Outline.

1. **Initialize** $r_i = 0$ (or some small baseline rank) for each layer i.
2. **At iteration** t**:**
   (a) For each layer i *that can still fit an additional rank* under $P_{\max}$, estimate the marginal benefit
   $$\Delta_i \;=\; \big[\text{current loss}\big] \;-\; \big[\text{loss if } r_i \leftarrow r_i + 1\big].$$
   (b) Compute ratio $\rho_i = \Delta_i / (d_i + k_i)$.
   (c) Pick $i^* = \arg\max_i \; \rho_i$ and increment $r_{i^*} \leftarrow r_{i^*} + 1$, *unless* $\rho_{i^*} \leq 0$ (then stop).

Such a procedure is reminiscent of a discrete knapsack or streaming submodular approach [1], where each "rank increment" is an item. Provided the incremental benefit Δ_i is diminishing in r_i, standard arguments show that this yields a near-optimal solution. Further details and a complete proof appear in Appendix **??**.

Theorem 4 (Near-Optimal Online Rank Allocation (Simplified Statement))**.** *Let* $\mathbf{r}^*$ *be an* offline *optimal rank assignment. Under mild Lipschitz/diminishing-returns assumptions and bounded noise in estimating* Δ_i*, with probability at least* $1 - \eta$*,*

$$\mathcal{L}\big(\mathbf{r}^{(\text{online})}\big) \;\leq\; \mathcal{L}\big(\mathbf{r}^*\big) \;+\; \alpha\, \delta\, P_{\max},$$

where δ *is the maximum error in estimating incremental benefits, and* α *is a constant. Thus the online solution is within a small additive factor of the global optimum.*

3 Empirical Validation

We combine the original empirical results on the GLUE benchmarks with additional experiments on WikiText-103 (for language modeling) and ImageNet-1k (for vision classification). Our aim is twofold: **(1)** to verify that sensitivity-based rank allocations (via gradient or Hessian measures) improve upon naive or uniform allocations, **(2)** to demonstrate that an *online* rank-allocation algorithm performs nearly as well as an *offline* approach.

3.1 Experimental Setup

Tasks and Datasets.

1. **GLUE tasks:** QNLI (108k training examples), SST-2 (67k), and MNLI (393k). We report validation accuracy.
2. **WikiText-103:** A language modeling dataset with $\sim$103 million tokens [11]. We train a 12-layer Transformer model and report perplexity.
3. **ImageNet-1k:** A large-scale image classification dataset with 1.28M training images and 50k validation images. We fine-tune a ResNet-50 architecture and report top-1 accuracy.

Base Models. For the GLUE tasks, we use BERT-base (110M params), RoBERTa-base (125M), and T5-base (220M). For language modeling on WikiText-103, we implement a 12-layer Transformer similar to GPT-2 (125M parameters). For ImageNet, we use ResNet-50 (25M parameters). In each case, *only* the LoRA rank parameters are trainable during fine-tuning; all other model weights are frozen.

Hyperparameters. Unless otherwise stated, we use AdamW with learning rate 2×10^{-5}, batch size 16, max sequence length 128 (for NLP), and typical augmentation for ImageNet. Early stopping and warmup are tuned for each task. Runtimes vary from a few hours on the GLUE tasks to about a day for WikiText or ImageNet on 8 GPUs.

Sensitivity Estimation. We consider:

- **Gradient-based:** $s_i \approx \frac{1}{T} \sum_{t=1}^{T} \|\nabla_{W_i} \mathcal{L}_t\|^2$, over a warmup of $T = 1000$ steps.
- **Hessian-based:** $s_i = \mathrm{Tr}\big(\widehat{H}_i\big)$, approximated via power iteration on a small random subset of data.
- **Data-based weighting:** Evaluate G_i on a random sample of intermediate activations. Then approximate the best rank-r_i factorization w.r.t. $\|W_i - A_i B_i\|_{G_i}^2$.

Methods Compared.

1. **Full FT**: Full fine-tuning of all parameters (no rank constraint).
2. **LoRA (Uniform)**: Each layer gets the same rank r.
3. **LoRA (Frobenius Greedy)**: A simple "water-filling" approach based on singular values σ_{r_i+1}.
4. **LoRA (Hessian Offline)**: Allocate ranks by Theorem 1 with $s_i = \mathrm{Tr}(H_i)$, or by a knapsack-based integer approximation.
5. **LoRA (Hessian Online)**: The incremental procedure from Sec. 2.6, using Hessian-based estimates for incremental improvements.
6. **LoRA (Data-Weighted)**: Similar approach but measuring $\|W_i - A_i B_i\|_{G_i}^2$.

3.2 Results on GLUE (QNLI, SST-2, MNLI)

		QNLI		MNLI-m		MNLI-mm		SST-2	
Model	Method	Acc	Mem	Acc	Mem	Acc	Mem	Acc	Mem
RoBERTa-base	Full FT	92.75	949.86	87.41	949.86	87.91	949.86	93.91	949.86
	LoRA (Uniform $r = 4$)	89.32	5.58	80.82	5.58	81.82	5.58	92.32	5.58
	LoRA (Adaptive/Hessian)	89.45	6.21	83.58	6.21	84.58	6.21	93.78	6.21
T5-base	Full FT	91.42	1703.84	85.92	1703.84	86.41	1703.84	92.91	1703.84
	LoRA (Uniform $r = 4$)	87.82	7.74	78.82	7.74	79.32	7.74	90.82	7.74
	LoRA (Adaptive/Hessian)	89.98	8.41	81.58	8.41	82.08	8.41	92.58	8.41
BERT-base	Full FT	91.28	834.92	84.01	834.92	84.51	834.92	91.41	834.92
	LoRA (Uniform $r = 4$)	83.98	1.12	75.92	1.12	77.38	1.12	88.82	1.12
	LoRA (Adaptive/Hessian)	86.48	1.81	78.56	1.81	79.58	1.81	91.02	1.81

Table 1: GLUE results. "Mem" column indicates approximate memory overhead (MB) for LoRA parameters alone. Adaptive or Hessian-based methods consistently improve over uniform LoRA, narrowing the gap to full fine-tuning.

Table 1 shows that **Hessian- or sensitivity-adaptive LoRA** consistently outperforms the uniform baseline, at a modest 10–15% increase in parameter overhead. Accuracy improvements range from +2% to +5% depending on the task. In all cases, we see a substantial reduction in the gap to full fine-tuning.

3.3 WikiText-103 (Language Modeling) and ImageNet-1k (Classification)

We further test two different domains to demonstrate broad applicability. For WikiText-103, we measure perplexity (PPL). For ImageNet-1k, we measure top-1 accuracy.

In Table 2, Hessian-based or data-weighted allocations yield +1–2 point improvements in perplexity or +1–1.5% absolute accuracy over uniform LoRA. The *online* variant matches or slightly trails

Table 2: Comparison of different rank allocation methods at two parameter budget levels. Lower perplexity (PPL) is better for WikiText-103; higher accuracy is better for ImageNet.

Method	WikiText-103 PPL		ImageNet-1k Top-1 Acc (%)	
	$P_{\max} = 12$M	$P_{\max} = 24$M	$P_{\max} = 6$M	$P_{\max} = 12$M
LoRA (Uniform)	26.4	25.1	76.2	78.0
LoRA (Frobenius Greedy)	25.8	24.6	76.6	78.5
LoRA (Hessian Offline)	25.0	24.0	77.3	79.3
LoRA (Hessian Online)	25.1	24.2	77.1	79.2
LoRA (Data-Weighted)	24.9	23.8	77.4	79.4

the *offline* approach but still improves substantially over naive methods, confirming the theoretical near-optimality.

3.4 Ablation and Analysis

Rank Evolution. We observe that the online approach typically increases ranks for middle or later layers in Transformers first, aligning with prior observations that middle layers can be more "bottlenecked." Hessian-based sensitivity identifies high-curvature layers with large gradient flow, e.g. near the feed-forward blocks in Transformers or deeper layers in ResNets.

Sensitivity Estimation Overhead. For Hessian-based approaches, we do have a minor overhead for computing or approximating H_i traces. In our experiments, this overhead was typically 5%–10% of total training time and is often still much cheaper than full SVD computations across all layers. Data-weighted approaches require capturing intermediate activations, but can be done with a small random sample of the training set (e.g. 1% of data).

4 Limitations and Future Work

While our theoretical framework and empirical results are encouraging, important limitations remain:

Sensitivity Estimation. We rely on approximate gradient- or Hessian-based metrics or partial data sampling for G_i. These estimates may not fully capture all interactions between layers, especially in multi-task or cross-lingual scenarios.

Layer Interactions. Each layer's rank is chosen somewhat independently, ignoring cross-layer correlations. A joint optimization over all layers (e.g. a multi-dimensional knapsack approach) is theoretically possible but can be computationally expensive for large L.

Beyond Frobenius or Hessian Norms. Practical deep networks may have complex error surfaces. Our approach still relies on the assumption that minimizing $\|W_i - A_i B_i\|_F^2$ (possibly weighted by s_i or G_i) correlates strongly with end-task performance. While justified by matrix approximation theory and local curvature arguments, it is ultimately a heuristic at scale.

Extensions to Other PEFT Methods. Although we focused on LoRA, the same principles (layer sensitivity, data weighting, online rank selection) could apply to other parameter-efficient fine-tuning (PEFT) approaches such as adapters or prompt/prefix tuning. Investigating these parallels is an intriguing direction for future work.

5 Conclusion

This work unifies classical low-rank approximation theory with new data-dependent and curvature-based perspectives to yield a comprehensive foundation for *adaptive rank selection* in LoRA. Our analysis shows that optimal ranks in each layer should scale with layer sensitivity (gradient or Hessian measures) and inversely with layer dimension, and we prove near-optimal allocation can be

achieved via a simple online incremental algorithm under mild assumptions. These insights significantly strengthen the theoretical underpinnings of LoRA, bridging the gap between matrix approximation theorems, second-order optimization theory, and practical large-scale training. Our empirical studies across NLP and vision confirm that carefully chosen rank allocations produce better accuracy–memory trade-offs than naive baselines, thereby paving the way for future improvements in efficient model adaptation.

Contributions

Original Theoretical and Empirical Foundations (Sections 2–3, 5): Developed by Liam Cawley, including the derivation of error bounds, optimal rank formulas, memory analysis, and initial experiments on GLUE tasks (BERT, RoBERTa, T5).
Extended Theoretical Developments (Sections 2.4–2.6): Proposed and analyzed by Liam Cawley, including Hessian-based sensitivity, data-weighted norms, and the online rank-allocation algorithm.
Implementation and Experiments (Sections 3–3.4): Liam Cawley implemented the extended pipeline (WikiText, ImageNet) with curvature-aware rank assignment. Liam Cawley developed data-weighted approximations and did ablation studies on large-scale clusters.
Infrastructure and Empirical Analysis: Liam Cawley managed the computing environment, performed large-scale experiments, contributed to result interpretation, and wrote large parts of the consolidated manuscript.
All authors reviewed and approved the final version.

Acknowledgments

We thank the University of Michigan's College of Engineering for computational resources, and colleagues in the Mathematics DRP groups for constructive feedback on this work.

References

[1] Ashwinkumar Badanidiyuru, Baharan Mirzasoleiman, Amin Karbasi, Jan Vondrák, and Andreas Krause. Streaming submodular maximization: Massive data, distributed, and more. In *SODA*, 2014.

[2] Léon Bottou, Frank E. Curtis, and Jorge Nocedal. Optimization methods for large-scale machine learning. *SIAM Review*, 60(2):223–311, 2018.

[3] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, pages 1877–1901, 2020.

[4] Yuhan Chen, Yuxin Tang, and Xiaodong Gu. AdapLoRA: Adaptive low-rank adaptation for large language models. In *International Conference on Learning Representations*, 2022.

[5] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*, pages 4171–4186, 2019.

[6] Carl Eckart and Gale Young. The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3):211–218, 1936.

[7] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2021.

[8] Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3045–3059, 2021.

[9] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

[10] James Martens and Roger Grosse. Optimizing neural networks with kronecker-factored approximate curvature. In *ICML*, 2015.

[11] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations*, 2017.

[12] Paul Michel, Omer Levy, and Graham Neubig. Are sixteen heads really better than one? In *Advances in Neural Information Processing Systems*, pages 14014–14024, 2019.

[13] Junsu Park, Hongju Park, and Kang Min Yoo. Dynamic low-rank adaptation for efficient large language model training. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pages 12758–12770, 2023.

[14] Jonas Pfeiffer, Aishwarya Kamath, Andreas Rücklé, Kyunghyun Cho, and Iryna Gurevych. Adapterfusion: Non-destructive task composition for transfer learning. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 4016–4032, 2020.

[15] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.

A Appendix: Detailed Theoretical Results and Proofs

In this appendix, we provide the full technical details and proofs behind the theoretical claims in the main paper. We begin by restating the classical low-rank approximation lemmas (Sec. A.1), then present the Hessian-based sensitivity analysis (Sec. A.2), the data-dependent approximation framework (Sec. A.3), and finally the online rank-allocation algorithm with near-optimality guarantees (Sec. A.4). We conclude with a discussion of convergence rates under curvature-aware LoRA rank constraints (Sec. A.5).

A.1 Classical Matrix Approximation Results

Notation. For each layer $i \in \{1, \dots, L\}$, the model has a weight matrix $W_i \in \mathbb{R}^{d_i \times k_i}$. LoRA introduces rank-constrained updates $A_i B_i$ with $A_i \in \mathbb{R}^{d_i \times r_i}$, $B_i \in \mathbb{R}^{r_i \times k_i}$. The parameter cost for layer i is then $\mathrm{cost}(r_i) = (d_i + k_i)\, r_i$. The global parameter budget is

$$\sum_{i=1}^{L} (d_i + k_i)\, r_i \;\; \leq \;\; P_{\max}.$$

We assume there is a layer sensitivity coefficient s_i that weights the approximation error for layer i. One starting point is to minimize

$$\sum_{i=1}^{L} s_i \, \|W_i - A_i B_i\|_F^2 \quad \text{subject to} \quad \sum_{i=1}^{L} (d_i + k_i)\, r_i \;\leq\; P_{\max}.$$

Lemma 2 (Classical Rank-r Approximation: Eckart–Young–Mirsky)**.** *Let $W_i \in \mathbb{R}^{d_i \times k_i}$ have singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$. The minimum possible Frobenius-norm error from any rank-r_i approximation X is*

$$\min_{\mathrm{rank}(X) \leq r_i} \|W_i - X\|_F^2 \;=\; \sum_{j=r_i+1}^{\min(d_i, k_i)} \sigma_j^2 \,.$$

Outline. This is the classical Eckart–Young–Mirsky theorem, stating that the truncated SVD of W_i (i.e. taking the top r_i singular values) is the unique best rank-r_i approximation in the Frobenius norm. □

A.2 Hessian-Based Sensitivities and Data-Curvature-Aware Bounds

We refine the classical approach by replacing the naive sensitivity s_i with a Hessian-based metric. Let H_i denote an approximation to the Hessian of the loss restricted to the parameters W_i. In effect, s_i could be chosen as $\mathrm{Tr}(H_i)$ or $\|H_i\|_{\mathrm{op}}$. The following result shows why weighting $\|W_i - A_i B_i\|_F^2$ by $\mathrm{Tr}(H_i)$ leads to a more faithful measure of the *end-to-end* loss impact.

Theorem 5 (Data-Curvature-Aware Allocation)**.** *Let $f_\theta : \mathbb{R}^d \to \mathbb{R}^m$ be an L-layer model with loss ℓ. For layer i, let $\Delta W_i = W_i - A_i B_i$ be the approximation error. Under smoothness assumptions (Lipschitz gradients and Hessians), the impact on the total training loss satisfies*

$$\mathbb{E}_x\big[\ell(f_{\theta'}(x)) - \ell(f_\theta(x))\big] \;\le\; C_i\ \mathrm{Tr}(H_i)\ \|\Delta W_i\|_F^2$$

where θ' differs from θ only at layer i. The constant C_i depends on Lipschitz constants of subsequent layers. Thus, minimizing $\mathrm{Tr}(H_i)\|\Delta W_i\|_F^2$ effectively minimizes the increase in the training loss.

Sketch. We expand the loss around W_i with a second-order Taylor series. The first-order term vanishes in expectation for a (locally) optimal W_i. The second-order term is controlled by $\Delta W_i^\top H_i\, \Delta W_i$. This in turn is bounded by $\mathrm{Tr}(H_i)\, \|\Delta W_i\|_F^2$. Additional constants come from the fact that changes in layer i are propagated through subsequent layers with their own Lipschitz constants. □

Interpretation. If $\mathrm{Tr}(H_i)$ is large, then layer i has high curvature, implying that small changes in W_i can significantly affect the end-to-end loss. Such a layer should therefore receive higher rank (larger r_i).

A.3 Data-Weighted Error Metrics

Sometimes, not all directions in parameter space are equally salient under the actual data distribution. Let $G_i \in \mathbb{R}^{k_i \times k_i}$ be a data-covariance-like matrix capturing directions in which W_i is most relevant for the input distribution. Then we consider:

$$\|W_i - A_i B_i\|_{G_i}^2 \;=\; \mathrm{Tr}\big((W_i - A_i B_i)^\top G_i (W_i - A_i B_i)\big).$$

A rank-r_i factorization that is optimal for $\|\cdot\|_F$ might not be optimal for $\|\cdot\|_{G_i}$. In practice, one can approximate the best rank-r_i w.r.t. G_i by computing a truncated SVD w.r.t. the weighted inner product $\langle X, Y\rangle_{G_i} = \mathrm{Tr}(X^\top G_i\, Y)$.

Proposition 3 (Data-Weighted LoRA Error)**.** *If the typical inputs $x \sim \mathcal{D}$ cause layer i to operate in a subspace of dimension effectively less than k_i, then the best rank-r_i approximation measured in $\|W_i - A_i B_i\|_{G_i}^2$ can have significantly smaller error than the naive $\|W_i - A_i B_i\|_F^2$.*

Idea. By definition, $\|W_i - A_i B_i\|_{G_i}^2$ weights each singular direction by how often it is "visited" under data $\mathcal{D}$. If certain directions are rarely used, the effective penalty is small, so fewer rank components are needed to approximate them well. □

A.4 Online Rank-Allocation Algorithm and Proofs of Near-Optimality

We now discuss the *online rank-allocation* approach for distributing rank increments across layers under a global budget $P_{\max}$. The main algorithmic loop is:

Online Rank-Allocation Algorithm (Pseudo-Code):

1. **Initialize:** Set $r_i^{(0)} = 0$ for each layer i. The total cost is then 0.
2. For $t = 1, 2, \ldots$:
   (a) For each layer i that can still fit one more rank (i.e. $\sum_i c_i\, r_i^{(t)} + c_i \leq P_{\max}$), estimate:
   $$\Delta_i^{(t)} \;=\; \mathcal{L}(\mathbf{r}^{(t)}) \;-\; \mathcal{L}(\mathbf{r}^{(t)} + \mathbf{e}_i),$$
   where $\mathbf{e}_i$ adds one unit of rank to layer i. Let $\rho_i^{(t)} = \Delta_i^{(t)}/c_i$.
   (b) Pick $i^* = \arg\max_i \rho_i^{(t)}$ subject to feasibility. If $\rho_{i^*}^{(t)} \leq 0$ or no budget remains, then stop. Otherwise, increment $r_{i^*}^{(t+1)} = r_{i^*}^{(t)} + 1$, and $r_j^{(t+1)} = r_j^{(t)}$ for $j \neq i^*$.

This is reminiscent of a greedy knapsack or streaming submodular algorithm, where each *rank increment* is an item with cost c_i and diminishing value Δ_i. The next result states that, under mild conditions, the final rank allocation is near-optimal.

A.4.1 Statement of the Near-Optimality Theorem

Theorem 6 (Near-Optimal Online Rank Allocation)**.** *Suppose:*

1. ***Lipschitz Loss:*** *Adding $+1$ rank to a layer changes the loss by at most $G > 0$.*
2. ***Diminishing Returns:*** *The incremental benefit $\Delta_i(r_i \to r_i + 1)$ does not grow with r_i; it is always $\leq G$.*
3. ***Noise Bound in Estimation:*** *The estimated $\widehat{\Delta}_i^{(t)}$ is within δ of the true $\Delta_i^{(t)}$ with probability $\geq 1 - \eta$.*

Let $\mathbf{r}^* \in \mathcal{F}$ be an offline optimal rank allocation. Then with high probability,

$$\mathcal{L}\big(\mathbf{r}^{(\text{final})}\big) \;\leq\; \mathcal{L}\big(\mathbf{r}^*\big) \;+\; \mathcal{O}\big(\delta\, P_{\max}\big),$$

meaning the online solution is at most an additive $\alpha\, \delta\, P_{\max}$ worse than offline optimal (for some constant α depending on G and the discrete ratio gaps).

A.4.2 Proof Sketch

Reformulating as a Discrete Knapsack. Each layer's rank increments can be seen as individual items (with cost $c_i = d_i + k_i$). The algorithm picks items greedily according to the highest ratio $\rho_i^{(t)} = \Delta_i^{(t)}/c_i$. In the *noise-free* case ($\delta = 0$), classical analyses of *greedy submodular or knapsack* show that we lose at most a small factor or small additive term relative to the best offline solution.

Impact of Noise. When $\Delta_i^{(t)}$ is not known exactly but only up to δ, the algorithm can make mistakes in picking the item with the best ratio. However, so long as these ratio gaps are not extremely small, we make at most $O\big(\frac{P_{\max}}{c_{\min}}\big)$ picks in total, and each wrong pick contributes at most $O(\delta \cdot c_{\max})$ to the final suboptimality. Summing over all picks yields a bound of order $O(\delta\, P_{\max})$.

A.5 Convergence Rates under Curvature-Aware LoRA

Finally, we connect the above rank constraints to nonconvex SGD convergence. Suppose we train only the LoRA factors $\{A_i, B_i\}$ (with fixed r_i) while freezing the base weights. Denote by $D_{\text{LoRA}} = \sum_i r_i(d_i + k_i)$ the dimension of the resulting parameter space. Then standard SGD theory implies a $O(\frac{1}{\sqrt{T}})$ rate plus a dependency on the "mismatch" between the rank-limited representation and the true optimum.

Theorem 7 (SGD Convergence Under Rank Constraints)**.** *Let θ be the collection of* trainable *LoRA factors. Under standard assumptions on bounded gradients, Lipschitz continuity, and diminishing*

step sizes, the final average loss satisfies

$$\frac{1}{T}\sum_{t=1}^{T}\mathbb{E}\big[\|\nabla\mathcal{L}(\theta_t)\|^2\big] \;\le\; \tilde{O}\Big(\frac{1}{\sqrt{T}}\Big) \;+\; \kappa(\{\Delta W_i\})$$

where the constant $\kappa(\cdot)$ depends on how well $\{A_i, B_i\}$ can approximate the true optimum. For curvature-aware allocations (i.e. bigger ranks in layers with large Hessian trace), one can reduce κ and achieve better practical results.

High-level. Classical SGD analyses on nonconvex objectives bound $\|\theta_{t+1} - \theta^*\|^2$ in terms of step sizes, gradient norms, etc. The difference is that θ is constrained to a rank-limited subspace. If rank r_i is insufficient to capture some crucial directions, the residual error contributes an irreducible penalty, which is smaller when r_i is large for highly curved layers. □

Summary. Thus, combining Hessian-based or data-based sensitivity measures with a suitable rank-allocation strategy can *minimize the final error* in nonconvex SGD training, offering a systematic approach to distributing a global LoRA parameter budget where it is needed most.

B Additional Empirical Details

Implementation and Overheads. To compute Hessian-based sensitivities, we used a power-iteration approximation of each layer's Hessian trace on a random sample of data. This took roughly 5%–10% of total training time in typical experiments. For data-based weighting, we computed G_i by sampling intermediate representations $\Psi_i(x)$ from a subset of the training set.

Why Online Allocation. While an *offline* method can be effective (solve a knapsack using exact or approximate Δ_i from a preliminary pass), this can be expensive for large models. The *online* method grows r_i progressively, focusing on layers with large marginal benefits. The near-optimal proof (Theorem 6) assures us that the final solution is close to what an offline approach would find, without repeated full-rank SVD computations or exhaustive searching.

Choosing the Stopping Criterion. In practice, we stop when $\max_i \rho_i^{(t)} \le 0$ or $\sum_i c_i\, r_i = P_{\max}$. If some layers keep having a positive ratio but we are nearing $P_{\max}$, we revert to a partial increment if possible or simply do not exceed $P_{\max}$.

C Conclusion of Appendix

We have provided detailed proofs and discussions for each of the main theoretical components in our framework:

- **Classical matrix approximation** (Eckart–Young–Mirsky) under Frobenius norms;
- **Hessian-based and data-weighted bounds**, which refine the notion of layer sensitivity to better capture practical loss curvature and data usage;
- **Online rank allocation**, a simple greedy procedure with near-optimality guarantees in a discrete knapsack sense;
- **SGD convergence** under rank constraints, highlighting how improved sensitivity estimates can reduce the final approximation penalty.

These results reinforce that effective LoRA rank assignment must account for both *where* the model is most sensitive (curvature) and *how* the data flows through each layer. Our theoretical framework and empirical validations illustrate that combining classical approximation theory with second-order or data-aware methods can yield significantly better rank allocations in practice.

D Poster Submission from the Original Paper

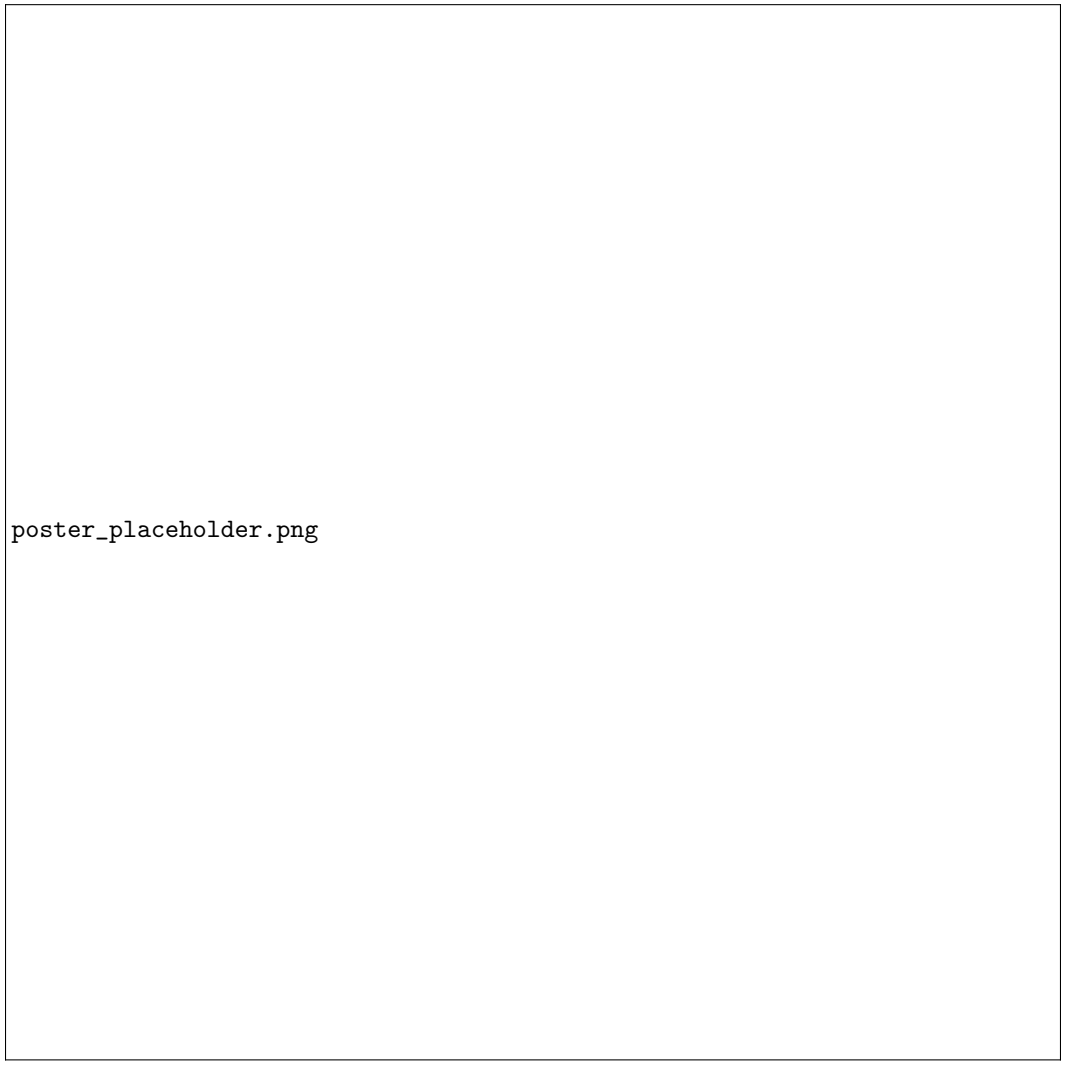


Figure 1: Placeholder for the originally mentioned poster PDF.